1. Start simple: Board representation + first test

Begin with the absolute minimum: a board that can be created.

Test (JUnit)

```
@Test
void newBoard_isEmpty() {
    Board board = new Board();
    assertEquals("         ", board.toString());
}
```

Data format:

Implementation

```
public class Board {
    private char[] cells = new char[9];

    public Board() {
        Arrays.fill(cells, ' ');
    }

    @Override
    public String toString() {
        return new String(cells);
    }
}
```

Data format:

👉Teaching point:

- Red → Green → Refactor
- Simplest possible implementation

---

2. Place a move

Test

```
@Test
void placeMove_updatesBoard() {
    Board board = new Board();
    board.place(0, 'X');

    assertEquals('X', board.getCell(0));
}
```

Data format:

Implementation

```
public void place(int index, char player) {
    cells[index] = player;
}

public char getCell(int index) {
    return cells[index];
}
```

Data format:

👉Teaching point:

- Introduce methods + encapsulation
- Avoid exposing internal array directly

---

3. Prevent invalid moves (introduce rules)

Test

```
@Test
void cannotPlaceOnOccupiedCell() {
    Board board = new Board();
    board.place(0, 'X');

    assertThrows(IllegalArgumentException.class, () -> {
        board.place(0, 'O');
    });
}
```

Data format:

Implementation

```
public void place(int index, char player) {
    if (cells[index] != ' ') {
        throw new IllegalArgumentException("Cell already taken");
    }
    cells[index] = player;
}
```

Data format:

👉Teaching point:

- Defensive programming
- Tests for failure cases

4. Winning logic (introduce domain logic)

Start with one winning case.

Test

```
@Test
void detectsRowWin() {
    Board board = new Board();
    board.place(0, 'X');
    board.place(1, 'X');
    board.place(2, 'X');

    assertTrue(board.hasWinner());
}
```

Data format:

Implementation (incremental)

```
public boolean hasWinner() {
    return cells[0] == 'X' && cells[1] == 'X' && cells[2] == 'X';
}
```

Data format:

Then expand:

```
private static final int[][] WINNING_COMBINATIONS = {
    {0,1,2}, {3,4,5}, {6,7,8},
    {0,3,6}, {1,4,7}, {2,5,8},
    {0,4,8}, {2,4,6}
};

public boolean hasWinner() {
    for (int[] combo : WINNING_COMBINATIONS) {
        char a = cells[combo[0]];
        if (a != ' ' &&
            a == cells[combo[1]] &&
            a == cells[combo[2]]) {
            return true;
        }
    }
    return false;
}
```

Data format:

👉Teaching point:

- Refactoring duplication into data structures
- Clean logic vs hardcoding

---

5. Introduce Game class (separation of concerns)

Now shift responsibility away from `Board`.

Test

```
@Test
void playersAlternateTurns() {
    Game game = new Game();

    game.play(0);
    game.play(1);

    assertEquals('X', game.getBoard().getCell(0));
    assertEquals('O', game.getBoard().getCell(1));
}
```

Data format:

Implementation

```
public class Game {
    private Board board = new Board();
    private char currentPlayer = 'X';

    public void play(int index) {
        board.place(index, currentPlayer);
        switchPlayer();
    }

    private void switchPlayer() {
        currentPlayer = (currentPlayer == 'X') ? 'O' : 'X';
    }

    public Board getBoard() {
        return board;
    }
}
```

Data format:

👉Teaching point:

- Single Responsibility Principle
- Game ≠ Board

---

6. Stop game when finished

Test

```java
@Test
void cannotPlayAfterGameWon() {
    Game game = new Game();

    game.play(0); // X
    game.play(3); // O
    game.play(1); // X
    game.play(4); // O
    game.play(2); // X wins

    assertThrows(IllegalStateException.class, () -> {
        game.play(5);
    });
}
```

Data format:

Implementation

```java
public void play(int index) {
    if (board.hasWinner()) {
        throw new IllegalStateException("Game already finished");
    }

    board.place(index, currentPlayer);
    switchPlayer();
}
```

Data format:

👉Teaching point:

- Business rules belong in Game
- Guard clauses

---

7. Detect draw

Test

```java
@Test
void detectsDraw() {
    Game game = new Game();

    // fill board without winner
    int[] moves = {0,1,2,4,3,5,7,6,8};
    for (int move : moves) {
        game.play(move);
    }

    assertTrue(game.isDraw());
}
```

Data format:

Implementation

```java
public boolean isDraw() {
    for (int i = 0; i < 9; i++) {
        if (board.getCell(i) == ' ') return false;
    }
    return !board.hasWinner();
}
```

Data format:

👉Teaching point:

- Combining conditions
- Expressive methods

---

8. Refactor to enums (cleaner design)

Replace `'X'` / `'O'` with:

```java
public enum Player {
    X, O
}
```

Data format:

Then update:

```java
private Player currentPlayer = Player.X;
```

Data format:

👉Teaching point:

- Avoid primitive obsession

- Strong typing

---

9. Optional stretch goals (if time allows)

- Add CLI interface
- Add AI (random moves)
- Parameterized tests for win conditions
- Introduce mocking (if UI layer added)

Suggested lecture flow

1. **Start naive** → build confidence
2. **Add tests first** → show safety net
3. **Break things intentionally** → show value of tests
4. **Refactor live** → show design evolution

If you want, I can turn this into:

- ↳ A **complete starter repo structure**
- ↳ A **scripted lecture plan with timing**
- Or ↳ **common student mistakes to trigger intentionally**

how to use enums

Enums are a perfect next step for your tic-tac-toe example—they solve a real problem (using `'X'` / `'O'` as raw chars) and introduce stronger typing in a very tangible way.

Let's build it progressively so you can live-code it.

---

1. Start with the problem

Right now you probably have:

```
char currentPlayer = 'X';
```

Data format:

Ask students:

- What if someone passes `'Z'`?
- What does `'X'` actually *mean*?

👉This sets up the need for enums.

---

2. Introduce a simple enum

Basic enum

```
public enum Player {
    X, O
}
```

Data format:

Use it

```
Player currentPlayer = Player.X;
```

Data format:

👉Teaching point:

- Enum values are **objects**, not primitives
- Safer than strings/chars

---

3. Replace char in Board

Before

```
private char[] cells;
```

Data format:

After

```
private Player[] cells;
```

Data format:

Constructor

```
public Board() {
    cells = new Player[9];
}
```

Data format:

👉Note: `null` now represents an empty cell

This is a great moment to discuss trade-offs.

---

4. Update methods

place()

```
public void place(int index, Player player) {
    if (cells[index] != null) {
        throw new IllegalArgumentException("Cell already taken");
    }
    cells[index] = player;
}
```

Data format:

getCell()

```
public Player getCell(int index) {
    return cells[index];
}
```

Data format:

---

5. Update tests (important!)

Before

```
assertEquals('X', board.getCell(0));
```

Data format:

After

```
assertEquals(Player.X, board.getCell(0));
```

Data format:

👉Teaching point:

- Compile errors guide refactoring

---

6. Add behavior to enum (this is the powerful part)

Enums aren't just constants—they can have logic.

Add method to switch player

```
public enum Player {
    X, O;

    public Player next() {
        return this == X ? O : X;
    }
}
```

Data format:

Use it in Game

```
currentPlayer = currentPlayer.next();
```

Data format:

👉This is a *great "aha moment"*:

- Behavior belongs with the data

---

7. Make Board printable again

Because now `toString()` will show `null`.

Improve it

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    for (Player p : cells) {
        sb.append(p == null ? " " : p);
    }
    return sb.toString();
}
```

Data format:

---

8. Improve win logic

Before (char-based)

```
char a = cells[combo[0]];
```

Data format:

After

```
Player a = cells[combo[0]];
if (a != null &&
    a == cells[combo[1]] &&
    a == cells[combo[2]]) {
    return true;
}
```

Data format:

👉Teaching point:

- Enums can be compared with `==` safely

---

9. Optional: add symbols (nice polish)

```
public enum Player {
    X("X"), O("O");

    private final String symbol;

    Player(String symbol) {
        this.symbol = symbol;
    }

    @Override
    public String toString() {
        return symbol;
    }
}
```

Data format:

👉Now UI logic is cleaner and extensible.

---

10. Nice teaching moments to highlight

Why enums are better

- Prevent invalid values
- Self-documenting
- Can contain behavior
- Compile-time safety

Common pitfalls

- Using `==` is OK for enums (unlike objects)
- `null` still exists → discuss alternatives:
  - Optional (advanced)
  - Special value like `EMPTY` (nice extension exercise)

---

11. Bonus exercise for students

Ask them to extend:

```
EMPTY
```

Data format:

```
public enum Player {
    X, O, EMPTY
}
```

Data format:

Then eliminate `null` entirely.

👉This sparks discussion about:

- Domain modeling
- Avoiding nulls

If you want, I can show you a **clean final version of the full game using enums + tests**, or a **bad version first** that you can refactor live (which usually makes the lesson much more engaging).

suggest a console UI application

A console UI is perfect here—it keeps things simple while letting you demonstrate **separation of concerns**, **input validation**, and even **testability trade-offs**.

I'd suggest building it in layers so students see how UI sits *on top* of your existing `Game` logic.

---

1. Start with the simplest possible loop

Begin with a "dumb" version just to get something running.

```
public class Main {
    public static void main(String[] args) {
        Game game = new Game();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println(game.getBoard());

            System.out.print("Enter move (0-8): ");
            int move = scanner.nextInt();

            game.play(move);
        }
    }
}
```

Data format:

👉Teaching point:

- "This works... but it's terrible" 😄
- No validation, no structure

---

2. Improve board rendering (make it visual)

Replace your `toString()` with something readable:

```
@Override
public String toString() {
    return String.format(
        "%s|%s|%s\n-+-+-\n%s|%s|%s\n-+-+-\n%s|%s|%s",
        cell(0), cell(1), cell(2),
        cell(3), cell(4), cell(5),
        cell(6), cell(7), cell(8)
    );
}

private String cell(int i) {
    return cells[i] == null ? " " : cells[i].toString();
}
```

Data format:

👉Output becomes:

```
X|O|
-+-+-
 |X|
-+-+-
 | |O
```

Data format:

---

3. Extract a ConsoleUI class (important design step)

Now show students how to separate concerns.

```java
public class ConsoleUI {
    private final Scanner scanner = new Scanner(System.in);

    public int askMove(Player player) {
        System.out.print("Player " + player + ", enter move (0-8): ");
        return scanner.nextInt();
    }

    public void showBoard(Board board) {
        System.out.println(board);
    }

    public void showWinner(Player player) {
        System.out.println("Player " + player + " wins!");
    }

    public void showDraw() {
        System.out.println("It's a draw!");
    }
}
```

Data format:

---

4. Refactor main to orchestrate

```java
public class Main {
    public static void main(String[] args) {
        Game game = new Game();
        ConsoleUI ui = new ConsoleUI();

        while (true) {
            ui.showBoard(game.getBoard());

            int move = ui.askMove(game.getCurrentPlayer());

            game.play(move);

            if (game.hasWinner()) {
                ui.showBoard(game.getBoard());
                ui.showWinner(game.getCurrentPlayer().next());
                break;
            }

            if (game.isDraw()) {
                ui.showBoard(game.getBoard());
                ui.showDraw();
                break;
            }
        }
    }
}
```

Data format:

👉Teaching point:

- `Main` = orchestration only
- UI + Game are independent

---

5. Add input validation (great live-coding moment)

Show how things break:

- user types `"abc"`
- user enters `99`

Then fix it.

```java
public int askMove(Player player) {
    while (true) {
        System.out.print("Player " + player + ", enter move (0-8): ");

        if (!scanner.hasNextInt()) {
            System.out.println("Invalid input!");
            scanner.next(); // consume bad input
            continue;
        }

        int move = scanner.nextInt();

        if (move < 0 || move > 8) {
            System.out.println("Move must be between 0 and 8.");
            continue;
        }

        return move;
    }
}
```

Data format:

---

6. Handle game exceptions cleanly

Right now `game.play()` might throw.

Wrap it:

```java
try {
    game.play(move);
} catch (IllegalArgumentException | IllegalStateException e) {
    System.out.println(e.getMessage());
}
```

Data format:

👉Teaching point:

- UI handles errors, Game enforces rules

---

7. Make it more user-friendly (nice polish)

Show positions:

```
0|1|2
-+-+-
3|4|5
-+-+-
6|7|8
```

Data format:

You can display this once at the start.

---

8. Optional: map input like "A1", "B2"

More advanced, fun exercise:

```java
public int parseMove(String input) {
    input = input.toUpperCase();

    int row = input.charAt(0) - 'A'; // A,B,C
    int col = input.charAt(1) - '1'; // 1,2,3

    return row * 3 + col;
}
```

Data format:

👉Great for:

- Strings
- ASCII math
- Validation logic

---

9. Testing discussion (important!)

Console apps are hard to test directly → perfect teaching moment.

Explain:

- Game is fully testable ✅
- ConsoleUI is harder ❌

Optional improvement:

- Inject `InputStream` / `PrintStream`
- Or wrap IO in interfaces

---

10. Final structure (clean architecture)

```
Game (logic, fully tested)
Board (data + rules)
Player (enum)

ConsoleUI (input/output)
Main (wiring)
```

Data format:

---

11. Nice live-coding flow

1. Start ugly (everything in `main`)
2. Extract UI
3. Add validation
4. Handle errors
5. Improve UX

Students will *feel* why structure matters.